

Languages:

Python

Databases:

CSV/Parquet Flat File Systems

Tools:

PyCharm / VS Code, Jupyter Notebook, GitHub, Jira

Technologies:

Pandas, NumPy, JSON Processing, Python File Handling (os, pathlib, shutil), Regular Expressions (Regex), Modular Programming, User-Defined Functions (UDFs), Error Handling (try-except), Data Validation Frameworks, ETL/ELT Pipeline Development, Data Preprocessing, Data Transformation, Metadata-Driven Processing (Control Files), Parquet File Processing, CSV Ingestion, Audit Logging, Data Quality Checks, Row-Count & Schema Reconciliation, Pipeline Orchestration, Aggregations & KPIs using Pandas, GroupBy Aggregations, Python Packaging & Modular Design, File Archival & Retention (ZIP compression), Source-to-Target Mapping (STTM), Data Lineage & Traceability

Roles & Responsibilities:

- Developed end-to-end **Python-based ETL workflows** for **RAW, PREPROCESSED, CURATED, and SEMANTIC layers** using **modules, packages, UDFs, control structures, file handling, and Pandas-based data pipelines** to automate ingestion, validation, preprocessing, transformation, and archival activities.
- Implemented **control-file-driven ingestion** using **JSON control file, Python dictionaries, and custom User-Defined Functions (UDFs)** to perform **schema validation, file existence checks, expected column validation, mandatory field checks, file-date extraction, datatype verification, and row-count reconciliation**—ensuring complete integrity before onboarding datasets.
- Utilized **Pandas (DataFrames), Python built-in functions, and vectorized operations** for high-performance ingestion of **CSV files** including **delimiter handling, UTF encoding, dtype assignment, header validation, and conversion to optimized Parquet format.**
- Designed preprocessing logic using **Python control structures (if-else, loops), list comprehensions, UDFs, and Pandas transformations** to perform **deduplication, null**

standardization, trimming, lower/upper case conversion, derived field generation, timestamp stamping, and feature enrichment before loading data into curated layers.

- Built curated transformation pipelines using **Pandas joins, merge operations, dictionary-based mappings, column alignment logic** to integrate **Claims, Customers, Policies, and Payments** dataframe.
- Engineered **semantic-level datasets** using **groupby aggregations, KPI derivation, functional programming (lambda, map), statistical summaries, pivot/unpivot logic**, and exported them using **Pandas I/O** and **file system utilities** into **Parquet/CSV** file formats.
- Created a robust **audit logging framework** using **Python file handling, CSV writer, datetime utilities**, and custom **UDFs** to consistently capture **ingestion metadata, timestamps, row counts, error messages, and execution status** for full audit traceability.
- Implemented **exception handling (try-except), custom error propagation, and safe-exit mechanisms** across ingestion, validation, preprocessing, and transformation layers to ensure pipeline stability and predictable error reporting.
- Designed **archival and retention logic** using **os module, shutil, pathlib, and ZIP compression utilities**, automating the movement of processed files and JSON control files into **timestamped archive directories** for compliance and lineage preservation.
- Wrote reusable **User-Defined Functions (UDFs)** for **filename date extraction, timestamp generation, column cleaning, schema verification, metadata parsing, and standardized transformation rules**—ensuring modularity and maintainability across pipeline components.
- Collaborated with **Data Engineers, BAs, and QA teams** to convert business logic into **Python transformation rules, Pandas operations, JSON-driven mappings, and domain-specific curated models**.